

Ingeniería del software de componentes y sistemas distribuidos

PR1 - Arquitecturas hexagonales y diseño de microservicios: una relación bien avenida

Índice

Descripción de la práctica.....	2
Ejercicio 1 (25 %).....	4
1.1. Operaciones del sistema (1 punto).....	4
1.2. Especificación de comandos (1 punto).....	8
1.3. Modelo general de dominio (0,5 puntos).....	13
Ejercicio 2 (15 %).....	14
2.1. Subdominios y servicios del sistema (0,75 puntos).....	14
2.2. Mapeo de operaciones (0,75 puntos).....	15
Ejercicio 3 (25 %).....	18
Ejercicio 4 (15 %).....	20
Ejercicio 5 (20 %).....	22

Descripción de la práctica

Tras una nueva ronda de reuniones con el grupo inversor, están muy satisfechos con el estilo arquitectónico basado en microservicios que hemos propuesto para Photo&Film4You en la primera actividad del curso (PEC1). Sin embargo, se nos ha pedido añadir nuevos requisitos y algunas aclaraciones/restricciones al catálogo de alquiler de material audiovisual:

- Para el alquiler de materiales se utilizarán los estados "planificados", "cancelados" y "devueltos". Los escenarios en los que el cliente no devuelve el material alquilado o lo devuelve en condiciones defectuosas permanecen fuera del alcance del proyecto.
- Las categorías y subcategorías no pueden ser modificadas ni eliminadas.
- Las descripciones de productos siempre pueden actualizarse; sin embargo, el precio del alquiler diario solo puede modificarse si no hay unidades de producto comprometidas para alquilar.
- Un producto solo puede ser dado de baja de nuestro sistema si no está comprometido actualmente para ningún alquiler. Esta acción desregistrará tanto el producto como todas las unidades asociadas. Desde la perspectiva del usuario, estos productos no están disponibles, es como si estuvieran eliminados. Sin embargo, no se eliminan de nuestra base de datos. En su lugar, se les marca el estado de "no activo".
- De manera similar, las unidades producto pueden designarse como "no operativas" exclusivamente cuando no están involucradas en ningún contrato de alquiler.
- En el ámbito tecnológico, nos piden que la persistencia de datos de la plataforma debe gestionarse usando JPA y PostgreSQL.

Por otro lado, se solicita a la empresa incorporar dos nuevos servicios para mejorar la experiencia del usuario. Estos están descritos en el enunciado general del proyecto de este curso:

- Gestión de cursos de formación audiovisual.
- Gestión de microcredenciales académicas respecto los cursos completados por un alumno.

Respecto el primer servicio, consiste en un módulo para gestionar cursos de formación audiovisual, en el que los usuarios puedan mejorar sus habilidades y aprovechar mejor el equipo disponible. Para simplificar este caso de uso:

- Se utilizará un Sistema de Gestión de Aprendizaje (LMS) externo de código abierto, que se personalizará para proporcionar gestión de cursos y ofrecer contenidos, foros, actividades y otros recursos de aprendizaje.
- No se debe realizar ningún trabajo adicional de desarrollo dentro de este LMS para este enunciado, pues nuestro enfoque principal está en el propio sistema de gestión académica. Concretamente: inscripciones de cursos, registros de cursos, perfiles de los miembros y una hoja detallada que indique el título, descripción, modo de impartición (presencial, online o híbrido), precio, idioma, ubicación, objetivos, metodología y otra información relevante.
- Los usuarios pueden explorar los cursos disponibles, consultar los detalles (metodología, objetivos...) y matricularse en un curso específico.
- El profesor responsable puede consultar a los estudiantes matriculados y modificar los detalles del curso, incluyendo el título, descripción, modalidad, objetivos, metodología, duración y

lenguaje. Además, el profesor es responsable de cerrar los informes de calificaciones al final del curso

- Los administradores pueden crear cursos (asociados a un profesor existente), abrir y cerrar periodos de inscripción, y cerrar registros de cursos, entre otras acciones. Por otro lado, respecto al servicio de generación de microcredenciales, la petición debe producirse en el momento que un alumno complete un curso y reciba la calificación. El sistema debe ser capaz de mostrar al administrador todas las peticiones pendientes de tratar, y procesar su aprobación de manera individual o por lotes. Una vez confirmada una microcredencial, el alumno es notificado y la credencial quedaría disponible en el expediente del alumno.
- Por simplificar: la generación de la microcredencial e incorporación en el expediente, será realizada por un sistema externo, quedando fuera del alcance de este proyecto.
- La petición de generación de la credencial se realizará de manera síncrona. La notificación al administrador para confirmar la credencial, y al alumno para indicar que ya está tramitada en su expediente, si que debe ser asíncrona.

En resumen, tal como explica el enunciado, el flujo de trabajo académico consta de las siguientes etapas: apertura de matrículas (administrador), cierre de matrículas (administrador), impartición de cursos (usuarios de instructores y estudiantes), cierre de informes de calificación (instructor), cierre de cursos, generación de microcredenciales y aceptación de éstas (administrador).

Ejercicio 1 (25 %)

Basándose en todos los requisitos establecidos en la **especificación Photo&Film4You**, especificar la arquitectura de microservicios de la aplicación indicando:

1.1. Operaciones del sistema (1 punto).

Usando el patrón CQRS, separa los **comandos** de las **consultas**:

Para cada uno de los **comandos** especifica:

- Actor
- Caso de uso
- Nombre del comando
- Descripción

Para cada uno de los **consultas** especifica:

- Actor
- Caso de uso
- Nombre de la consulta
- Descripción

NOTA: Debe usarse una nomenclatura "Lower Camel Case" para los nombres de comandos y consultas (<https://wiki.c2.com/?LowerCamelCase>).

Comandos:

Actor	Caso de uso	Comando	Descripción
Administrador	Dar de alta una sección/subsección	<i>createCategory()</i>	Crear una sección/subsección en el catálogo.
Administrador	Dar de alta un producto	<i>createProduct()</i>	Crear un producto dentro del catálogo.
Administrador	Modificar la descripción de un producto	<i>updateProductDescription()</i>	Modificar la descripción de un producto dentro del catálogo
Administrador	Modificar el precio diario de alquiler de un producto	<i>updateProductDailyPrice()</i>	Modificar el precio diario de alquiler de un producto si no hay unidades comprometidas

Administrador	Dar de baja un producto	<i>deleteProduct()</i>	Marca un producto y sus unidades como “no activo” si no está comprometido para alquiler.
Administrador	Dar de alta una unidad de un producto	<i>createItem()</i>	Crear una unidad de un producto dentro del catálogo.
Administrador	Establecer una unidad como operativa	<i>setOperationalItem()</i>	Establece una unidad de un producto como operativa que ya puede volver a ser alquilada.
Administrador	Establecer una unidad como no operativa	<i>setNotOperationalItem()</i>	Marca una unidad como no operativa exclusivamente cuando no está en un contrato de alquiler.
Cliente	Planificar un alquiler	<i>createRental()</i>	Registra un contrato de alquiler de material pasándolo al estado “planificado”.
Cliente / Administrador	Cancelar un alquiler	<i>cancelRental()</i>	Cambia el estado de un alquiler a “cancelado” liberando el material.
Administrador	Devolver material alquilado	<i>returnRental()</i>	Marca un contrato de alquiler con el estado “devuelto”.
Administrador	Crear curso	<i>createCourse()</i>	Crea un nuevo curso de formación y lo asocia a un profesor existente.
Administrador	Abrir periodo de inscripción	<i>openEnrollmentPeriod()</i>	Abre el plazo de inscripción a los cursos.
Administrador	Cerrar periodo de inscripción	<i>closeEnrollmentPeriod()</i>	Cierra el plazo de inscripción a los cursos.
Usuario registrado	Matricularse en un curso	<i>enrollInCourse()</i>	Registra la inscripción de un usuario en un

			curso específico.
Instructor	Cerrar informe de calificaciones	<i>closeGradingReport()</i>	Cierra las calificaciones de los alumnos, lo cual desencadena la petición de microcredenciales.
Instructor	Modificar detalles del curso	<i>updateCourse()</i>	Modifica los detalles del curso, incluyendo el título, descripción, modalidad, objetivos, metodología, duración y lenguaje.
Administrador	Cerrar curso	<i>closeCourse()</i>	Cierra un curso una vez finalizado completamente.
Administrador	Aprobar microcredenciales	<i>approveMicrocredentials()</i>	Procesa la aprobación de las peticiones de credenciales (individual o por lotes).

Consultas:

Actor	Caso de uso	Comando	Descripción
Usuario no registrado	Consultar las secciones/subsecciones disponibles	<i>getCategoriesByCriteria()</i>	Consultar las secciones y subsecciones disponibles en el catálogo por criterios.
Usuario no registrado	Explorar catálogo de productos	<i>getProductsByCriteria()</i>	Busca y lista productos del catálogo que están activos y disponibles.
Usuario registrado	Consultar historial de alquileres.	<i>getRentalsByCustomer()</i>	Devuelve el historial de alquileres de un cliente.
Usuario registrado	Explorar cursos disponibles	<i>getAvailableCourses()</i>	Busca y lista los cursos de

			formación disponibles para matricularse.
Usuario registrado	Consultar detalles de un curso	<i>getCourseDetails()</i>	Muestra información detallada de un curso (metodología, objetivos, precio, etc.).
Instructor	Consultar estudiantes matriculados.	<i>getEnrolledStudents()</i>	Lista los alumnos que se han inscrito en un curso impartido por el instructor.
Administrador	Listar todos los cursos	<i>getAllCourses()</i>	Muestra el estado de todos los cursos para su administración y gestión.
Administrador	Consultar peticiones de microcredenciales.	<i>getPendingMicrocredentials()</i>	Muestra al administrador todas las peticiones de credenciales pendientes de tratar.
Usuario registrado	Consultar perfil académico	<i>getStudentProfile()</i>	Recupera el perfil del miembro y los registros de cursos completados o en curso.
Usuario registrado	Consultar microcredenciales del alumno	<i>getStudentMicrocredentials()</i>	Consulta las microcredenciales aprobadas que constan en el expediente del alumno.

1.2. Especificación de comandos (1 punto).

Para cada comando, especifica sus parámetros, su valor de retorno, sus precondiciones y sus postcondiciones.

- **createCategory**

Operación: *createCategory(name, description, parentCategoryId)*

Retorno: *categoryId*

Precondiciones:

- No existe previamente ninguna categoría con ese nombre.
- Si se especifica la categoría padre (*parentCategoryId*), ésta debe existir.

Postcondiciones:

- La categoría se ha creado.
- Si se especifica la categoría padre, se crea una relación padre/hijo entre ambas.

- **createProduct**

Operación: *createProduct(categoryId, name, description, brand, model, dailyPrice)*

Retorno: *productId*

Precondiciones:

- La categoría especificada debe existir en el sistema.
- No existe previamente un producto activo con el mismo nombre exacto.

Postcondiciones:

- El producto se crea sin ninguna unidad como producto dentro de la categoría especificada

- **updateProductDescription**

Operación: *updateProductDescription(productId, newDescription)*

Retorno: True/False

Precondiciones:

- El producto debe existir en la base de datos.

Postcondiciones:

- La descripción del producto se ha actualizado con el nuevo valor introducido.

- **updateProductDailyPrice**

Operación: *updateProductDailyPrice(productId, newDailyPrice)*

Retorno: True/False

Precondiciones:

- El producto debe existir.
- No existe ninguna unidad de este producto que esté actualmente comprometida para alquilar.

Postcondiciones:

- El precio de alquiler diario del producto ha sido modificado.

- **deleteProduct**

Operación: *deleteProduct(productId)*

Retorno: True/False

Precondiciones:

- El producto debe existir en el sistema.
- El producto no está comprometido actualmente para ningún alquiler.

Postcondiciones:

- El producto no se elimina de la base de datos, sino que se marca con el estado de "no activo".
- Todas las unidades asociadas a este producto cambian su estado a "no activo".

- **createItem**

Operación: *createItem(productId, serialNumber)*

Retorno: itemId

Precondiciones:

- El producto al que se asocia debe existir.
- No debe existir otra unidad registrada con el mismo número de serie.

Postcondiciones:

- Se crea una unidad del producto en estado "operativa" asociada al producto y sin ninguna reserva de alquiler asociada.
- Se notifica por correo electrónico de la nueva unidad disponible a todos los usuarios que tengan una alerta de disponibilidad sobre ese producto, independientemente de las fechas de la alerta.

- **setOperationalItem**

Operación: *setNotOperationalItem(itemId)*

Retorno: True/False

Precondiciones:

- La unidad del producto debe existir y estar en estado "no operativa".

Postcondiciones:

- La unidad del producto pasa a estar en estado "operativa" y puede volver a ser alquilada.

- **setNotOperationalItem**

Operación: *setNotOperationalItem(itemId)*

Retorno: True/False

Precondiciones:

- La unidad del producto debe existir y estar en estado "operativa".
- La unidad no está involucrada actualmente en ningún contrato de alquiler.

Postcondiciones:

- El estado de la unidad cambia a "no operativa" y no puede ser alquilada.

- **createRental**

Operación: *createRental(customerId, itemIdsList, startDate, endDate)*

Retorno: rentalId

Precondiciones:

- El cliente debe existir en el sistema.
- Todas las unidades solicitadas deben existir, estar activas y ser operativas.

- Las unidades solicitadas no deben estar comprometidas en otros alquileres que se solapen con las fechas indicadas.

Postcondiciones:

- Se crea un contrato de alquiler asociado al cliente y a los artículos.
- El estado del alquiler queda establecido como "planificado".
- Las unidades quedan comprometidas para las fechas solicitadas.

- **cancelRental**

Operación: *cancelRental(rentalId)*

Retorno: True/False

Precondiciones:

- El contrato de alquiler debe existir.
- El estado actual del alquiler debe ser "planificado".

Postcondiciones:

- El estado del alquiler pasa a "cancelado".
- Las unidades de producto asociadas al contrato quedan liberadas de su compromiso de fechas.

- **returnRental**

Operación: *returnRental(rentalId)*

Retorno: True/False

Precondiciones:

- El contrato de alquiler debe existir.
- El estado actual del alquiler debe ser "planificado" (o un estado en curso, según la implementación del motor de estados temporal).

Postcondiciones:

- El estado del alquiler pasa a "devuelto".
- Las unidades de producto se liberan y vuelven a estar disponibles.

- **createCourse**

Operación: *createCourse(title, description, modality, price, language, location, objectives, methodology, duration, instructorId)*

Retorno: *courseId*

Precondiciones:

- El profesor (*instructorId*) debe existir en el sistema.

Postcondiciones:

- Se ha creado un nuevo curso con la información detallada.
- El curso queda asociado al instructor especificado.

- **openEnrollmentPeriod**

Operación: *openEnrollmentPeriod(courseId, startDate, endDate)*

Retorno: True/False

Precondiciones:

- El curso debe existir.

- El curso no debe tener un periodo de inscripción abierto actualmente ni estar ya cerrado definitivamente.

Postcondiciones:

- El periodo de matriculación del curso se marca como abierto, permitiendo la recepción de inscripciones.

- **closeEnrollmentPeriod**

Operación: *closeEnrollmentPeriod(courseId)*

Retorno: True/False

Precondiciones:

- El curso debe existir.
- El curso debe tener su periodo de inscripción actualmente abierto.

Postcondiciones:

- El periodo de matriculación se cierra. No se admiten nuevas inscripciones.

- **enrollInCourse**

Operación: *enrollInCourse(studentId, courseId)*

Retorno: *enrollmentId*

Precondiciones:

- El usuario y el curso deben existir.
- El periodo de inscripción del curso debe estar abierto.

Postcondiciones:

- Se registra la inscripción del alumno en el curso.

- **closeGradingReport**

Operación: *closeGradingReport(courseId, studentGradesList)*

Retorno: True/False

Precondiciones:

- El curso debe existir y el periodo de inscripción debe estar cerrado.
- El actor debe ser el profesor responsable de impartir el curso.

Postcondiciones:

- Las calificaciones del curso quedan registradas de manera definitiva en el sistema de gestión.
- Se dispara la petición de microcredenciales para aquellos alumnos que cumplan los requisitos de aprobación.

- **updateCourse**

Operación: *updateCourse(courseId, title, description, modality, objectives, methodology, duration, language)*

Retorno: True/False

Precondiciones:

- El curso debe existir.
- El actor que solicita la modificación debe ser el profesor responsable del curso o un administrador.

Postcondiciones:

- La información y detalles del curso han sido actualizados.

- **closeCourse**

Operación: *closeCourse(courseId)*

Retorno: True/False

Precondiciones:

- El curso debe existir.
- Los informes de calificaciones deben haber sido cerrados previamente por el profesor.

Postcondiciones:

- Los registros del curso quedan cerrados permanentemente (marcado como finalizado).

- **approveMicrocredentials**

Operación: *approveMicrocredentials(requestIdsList)*

Retorno: True/False

Precondiciones:

- Las peticiones indicadas en la lista deben existir y estar en estado "pendiente".

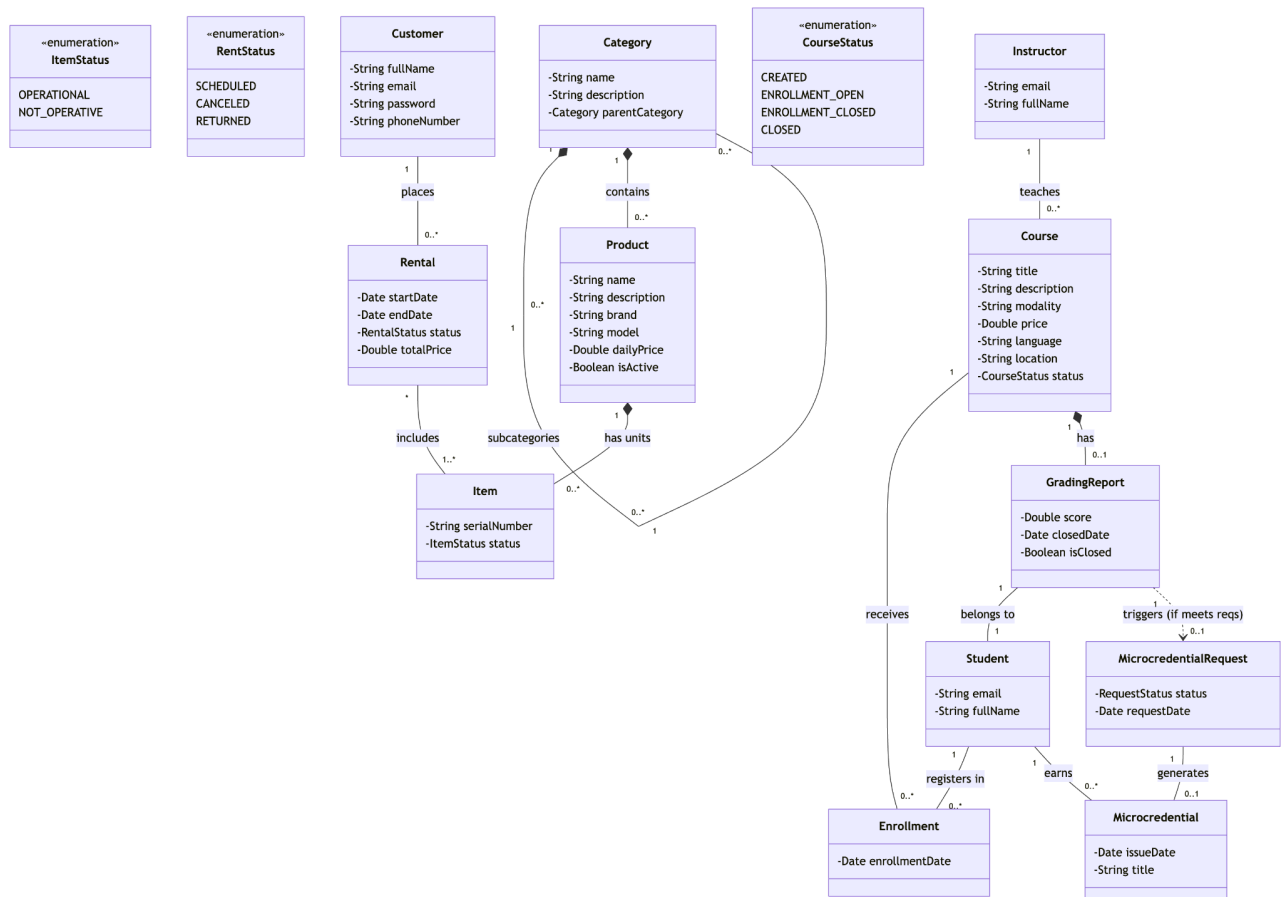
Postcondiciones:

- Las peticiones cambian su estado a "aprobadas".
- Se notifica de manera asíncrona al administrador confirmando la gestión.
- Se notifica de manera asíncrona al alumno indicando que su credencial está tramitada y disponible en su expediente.

1.3. Modelo general de dominio (0,5 puntos).

Diagrama de alto nivel del modelo de dominio.

NOTA: Puedes encontrar un ejemplo del diagrama solicitado en el libro "Microservices Patterns", capítulo 2: "Estrategias de descomposición", figura 2.7: "Las clases clave en el modelo de dominio FTGO".

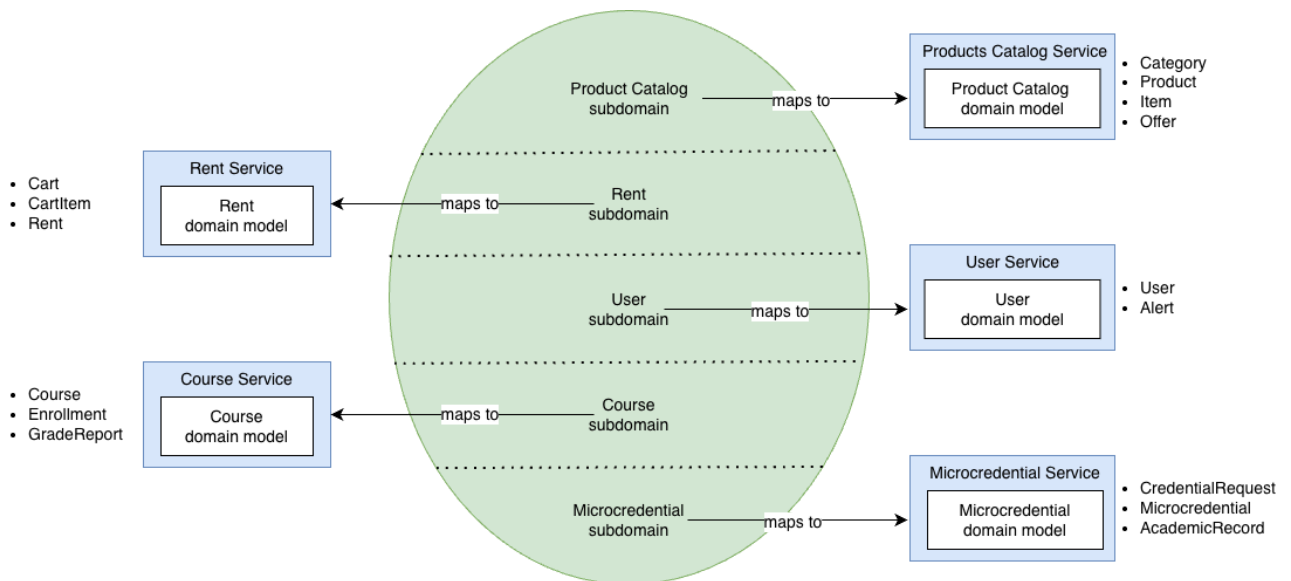


Ejercicio 2 (15 %)

Basándose en la definición de las operaciones del sistema y el modelo de dominio del ejercicio anterior, realizar la descomposición de los diferentes **microservicios** de la aplicación:

2.1. Subdominios y servicios del sistema (0,75 puntos)

Identificar los subdominios y sus servicios necesarios, utilizando una descomposición basada en el patrón de subdominios.



- **Product Catalog Service:** Gestión del catálogo de material para alquilar y las ofertas de productos de segunda mano.
- **Rent Service:** Gestión de la cesta de compra y las peticiones de alquiler realizadas por los usuarios.
- **User Service:** Gestión del registro de usuarios y las alertas asociadas.
- **Course Service:** Gestión de los cursos de formación audiovisual y del proceso de matriculación.
- **Microcredential Service:** Gestión del ciclo de vida de las certificaciones académicas. Está al margen del servicio de cursos para permitir que evolucione de forma independiente y para facilitar la integración con el sistema externo mencionado en el enunciado.
- **Notification Service:** Servicio transversal de notificaciones. En el caso especificado, permitirá notificar a los usuarios con alertas activas de la disponibilidad de algún elemento del catálogo.
- **Payment Service:** Gestión de los pagos. No se ha definido ningún subdominio, puesto que no se quiere tener ninguna información del pago.

2.2. Mapeo de operaciones (0,75 puntos)

Realizar el mapeo entre las operaciones del sistema y los servicios identificados, e identificar las colaboraciones que requieren algunos de los servicios restantes.

Servicio	Operaciones	Colaboradores
Product Catalog Service	API → Comandos: • <i>createCategory()</i> • <i>createProduct()</i> • <i>updateProductDescription()</i> • <i>updateProductDailyPrice()</i> • <i>deleteProduct()</i> • <i>createItem()</i> • <i>setNotOperationalItem()</i> • <i>setOperationalItem()</i> API → Consultas: • <i>getCategoriesByCriteria()</i> • <i>getProductsByCriteria()</i> • <i>getProductDetails()</i> • <i>getItemDetails()</i> • <i>getRentAvailableItems()</i> • <i>getOffersByUser()</i>	Rent Service: • <i>getRentReservations()</i> - Valida si se puede desactivar un item. • <i>hasActiveReservations()</i> - Comprueba si se puede borrar un producto o cambiar precio. Notification Service: • <i>notifyItemAvailable()</i> - Manda una notificación para marcar al item como "operativo".
Rent Service	API → Comandos: • <i>addCart()</i> • <i>addItemToCart()</i> • <i>deleteItemFromCart()</i> • <i>cancelRent()</i> • <i>checkoutRent()</i> API → Consultas: • <i>getRent()</i> • <i>getRentsByUser()</i> • <i>getCart()</i> Operaciones internas: • <i>getRentReservations()</i> • <i>hasActiveReservations()</i>	Payment Service: • <i>authorizePayment()</i> - Autoriza el pago al llamar al comando <i>checkoutRent</i>. Products Catalog Service: • <i>getItemDetails()</i> - Obtiene el precio del artículo y valida su existencia.
User Service	API → Comandos: • <i>signUp()</i> • <i>updateUserProfile()</i> • <i>getUserProfile()</i> • <i>createAlert()</i> • <i>dropAlert()</i> • <i>signIn()</i> • <i>logout()</i>	-

	API → Consultas: • <i>getAlertsByUser()</i> • <i>getAlert()</i> • <i>getUsersToAlert()</i>	
Course Service	API → Comandos: • <i>createCourse()</i> • <i>updateCourse()</i> • <i>openEnrollmentPeriod()</i> • <i>closeEnrollmentPeriod()</i> • <i>enrollInCourse()</i> • <i>closeGradingReport()</i> • <i>closeCourse()</i> API → Consultas: • <i>getAvailableCourses()</i> • <i>getCourseDetails()</i> • <i>getEnrolledStudents()</i> • <i>getAllCourses()</i> Operaciones internas: • <i>verifyCourseActive()</i>	Microcredentials Service: • <i>requestMicrocredential()</i> - Se ejecuta al llamar a <i>closeGradingReport()</i>.
Microcredential Service	API → Comandos: • <i>approveMicrocredentials()</i> API → Consultas: • <i>getPendingMicrocredentials()</i> • <i>getStudentMicrocredentials()</i> Operaciones internas: • <i>handleCourseGradedEvent()</i>	Notification Service: • <i>sendAdminPendingApprovalAlert()</i> - Notifica a los administradores que hay nuevas peticiones de microcredenciales esperando su revisión tras el cierre de actas. • <i>sendStudentCredentialApprovedAlert()</i> - Notifica al alumno de que un administrador ha aprobado su credencial.
Notification Service	Operaciones internas: • <i>sendItemAvailable()</i> • <i>sendAdminPendingApprovalAlert()</i> • <i>sendStudentCredentialApprovedAlert()</i>	User Service: • <i>getUsersToAlert()</i> - Obtiene la lista de usuarios que tienen configurada una alerta activa para un producto específico con el fin de notificarles cuando vuelva a estar disponible.
Payment Service	Operaciones internas: • <i>authorizePayment()</i>	-

Durante la descomposición podemos ver la necesidad de añadir operaciones internas que los microservicios tienen que ofrecer y no habíamos detectado en primera instancia.

Ejemplos:

- Los métodos de *User Service* son esenciales para que el usuario pueda interactuar con la plataforma. Cuenta con acciones como registrarse (*signUp*), iniciar y cerrar sesión (*signIn / logOut*), actualizar su perfil (*updateUserProfile*) y crear y eliminar alertas (*createAlert / dropAlert*).
- Los métodos de *Notification Service* permiten enviar alertas que se desencadenan a partir de una acción previa. Para que los usuarios reciban dichas alertas correctamente, creamos varios métodos en función de la naturaleza de la notificación, como *sendItemAvailable* para indicarle al usuario que un artículo vuelve a estar disponible, *sendAdminPendingApprovalAlert* para notificar al administrador que tiene una petición pendiente de revisar y *sendStudentCredentialApprovedAlert* para avisar a un alumno de que su microcredencial ha sido aprobada.
- El método *authorizePayment* de *Payment Service* sirve para autorizar el pago de un artículo. Abstrae toda la lógica de pasarelas de pago y facturación para permitir que otros subdominios como *Rent Service* puedan delegar la autorización y el procesamiento de los cobros en el momento de confirmar un alquiler.

Ejercicio 3 (25 %)

Basándonos en la descomposición realizada en el ejercicio anterior, detalla el diseño del servicio de cursos, teniendo en cuenta:

- El diseño debe realizarse utilizando una **arquitectura hexagonal**.
 - El diagrama debe representar los **protocolos de comunicación** con el servicio (REST, etc.) y los **métodos y atributos** de las diferentes clases del servicio (no es necesario indicar los tipos).
-

En primer lugar, detallamos los *driving adapters*, los cuales son los puntos de entrada a nuestro servicio. En este caso, traducen las peticiones HTTP REST externas a llamadas a nuestro propio servicio. *AcademicsRESTController* es el controlador que se encarga de exponer la API RESTful y delegar el procesamiento de las peticiones a *AcademicsService*.

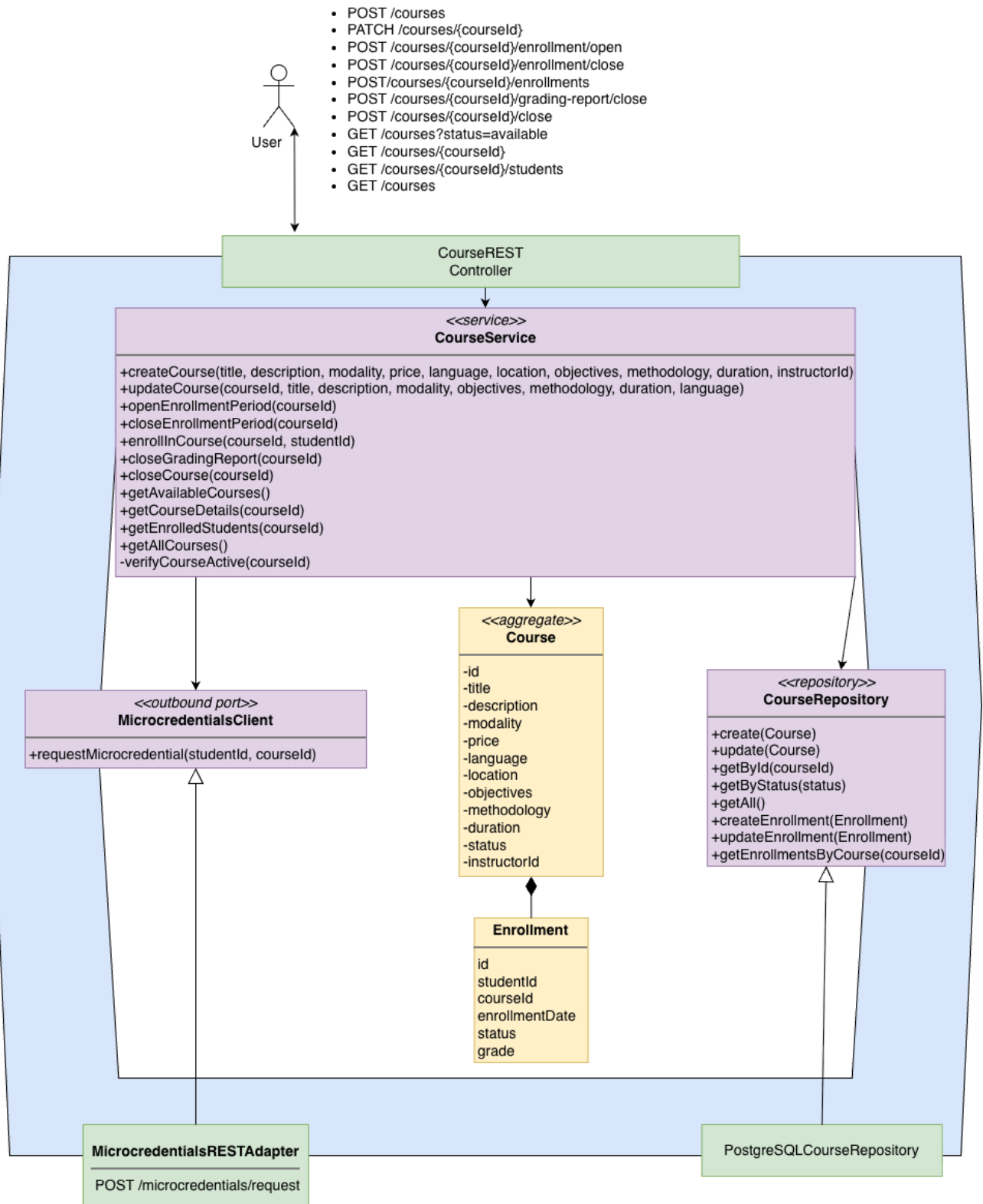
AcademicsService contiene la lógica de negocio y las interfaces (puertos) que definen cómo el servicio interactúa con el exterior. No tiene dependencias de ningún framework tecnológico externo, ya que implementa la lógica de negocio orquestando los comandos y consultas especificadas en el mapeo.

En cuanto a los modelos del dominio, encontramos *Course* (agregado principal), que contiene toda la información del curso y su estado. A su vez, se compone de la entidad *Enrollment*: la cual representa la matrícula de un alumno en un curso específico.

Los *driven adapters* implementan las interfaces (puertos de salida) definidas en el núcleo para comunicarse con la infraestructura, que pueden ser bases de datos o servicios externos. En nuestro caso, identificamos a *CourseRepository* como puerto de salida, ya que es una interfaz que define los métodos de persistencia en la base de datos. El enunciado especifica que se realice en PostgreSQL, por lo que creamos el adaptador secundario *PostgreSQLCourseRepository* para la Implementación concreta.

MicrocredentialsClient es otro puerto de salida, el cual actúa como una interfaz para la comunicación con el servicio externo de microcredenciales. Como adaptador secundario para este servicio, creamos *MicrocredentialsRESTAdapter* para implementar la llamada síncrona con la petición para solicitar una microcredencial.

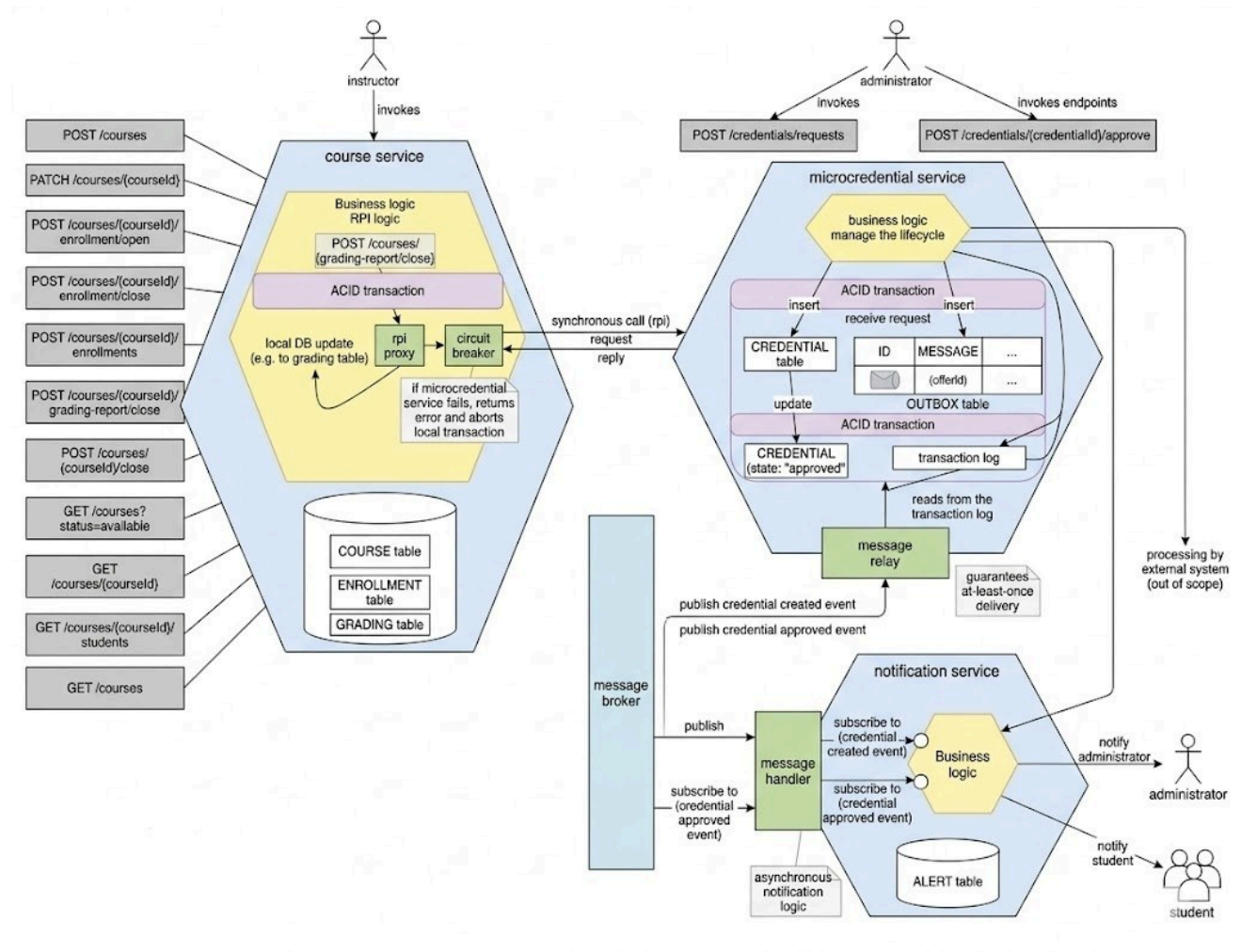
El resultado es el siguiente:



Ejercicio 4 (15 %)

¿Qué patrón o patrones aplicarías para gestionar el cierre del curso, la finalización de informes de calificaciones y la generación de microcredenciales, tal y como se describe en la documentación del proyecto?

Representa en un diagrama la solución de este enfoque, utilizando el patrón o patrones de microservicios más apropiados. Justifica tus decisiones.



Los patrones para resolver la problemática planteada es una combinación entre **Remote Procedure Invocation (RPI)**, **Circuit Breaker**, **Messaging**, y **Transactional Outbox**.

Cuando se produce el cierre de actas, se realiza una petición de credencial, la cual tiene que ser síncrona. El patrón **RPI** es útil cuando un cliente invoca un servicio esperando una respuesta inmediata (ej. REST/HTTP), por lo que se adapta perfectamente a la necesidad que tiene *Course Service* para solicitar la credencial.

No obstante, Richardson advierte que el patrón RPI reduce la disponibilidad, ya que si el servicio B cae, el servicio A también. Como la petición es síncrona, si el *Microcredential Service* falla, *Course Service* debe usar el patrón **Circuit Breaker** para devolver un error controlado al profesor y abortar la transacción local del cierre de actas, evitando así bloqueos en la base de datos.

Por otra parte, el enunciado exige que las notificaciones a administradores y alumnos sean asíncronas, por lo que el patrón **Messaging** mediante un canal *publish-subscribe* nos ayudará a cumplir este requisito. Richardson recomienda la mensajería asíncrona mediante un *Message Broker* para desacoplar servicios. Aplicado a nuestro caso, *Microcredential Service* publicará mensajes y *Notification Service* estará suscrito a ellos.

Cuando el administrador aprueba una credencial, el servicio debe hacer dos cosas: actualizar la base de datos para marcar la credencial como "aprobada" y enviar el mensaje al *broker*. No obstante, si la base de datos se guarda pero el *broker* se cae un milisegundo después, el alumno nunca recibirá la alerta. Para evitar esta casuística, podemos usar el patrón *Transactional Outbox*, ya que en él el servicio guarda el cambio de estado y el evento de notificación en una tabla *outbox* en la misma transacción local de base de datos. Luego, un proceso en segundo plano (*message relay*) lee la tabla *outbox* y envía el mensaje al *broker*. De este modo, garantiza una entrega "al menos una vez" (*at-least-once delivery*).

Ejercicio 5 (20 %)

El exitoso lanzamiento del proyecto **Photo&Film4You** ha despertado el interés de **Koursera**, una empresa consolidada conocida por ofrecer cursos online sobre una amplia variedad de temas. Koursera está explorando la creación de nuevos cursos centrados en cine y fotografía, donde los estudiantes tendrían la opción de alquilar equipos profesionales directamente a Photo&Film4You para seguir las lecciones prácticas.

Koursera quiere integrar su plataforma con la de **Photo&Film4You** para ofrecer a sus estudiantes la posibilidad de reservar y gestionar alquileres de equipos directamente desde su plataforma. Si se llega a un acuerdo, esto representaría una oportunidad para llegar a un público más amplio dentro de la industria cinematográfica independiente, a cambio de una pequeña comisión por cada transacción generada a través de **Koursera**.

La dinámica operativa implicaría que, al gestionar una grabación dentro de **Koursera**, los usuarios podrían acceder a una sección de alquiler de equipos. A partir de ahí, **Koursera** consultaría el catálogo de **Photo&Film4You** en tiempo real, junto con otros proveedores, y mostraría los productos disponibles. Si un usuario muestra interés en cualquiera de los modelos de alquiler disponibles en **Photo&Film4You**, la reserva se realizará directamente desde **Koursera**, y el procesamiento de pagos y las actualizaciones de inventario se gestionarán automáticamente por nuestra plataforma.

Aunque la dirección ve esta propuesta con buenos ojos, el Director de Tecnología (CTO) tiene algunas reservas. Durante una reunión técnica inicial con el **CTO de Koursera**, se observó que su solicitud se basa en servicios que requerirían consultar el catálogo de productos **Photo&Film4You**, reservas y gestión de alquileres. Sin embargo, durante las primeras pruebas de integración se detectaron problemas críticos: los alquileres cancelados no siempre liberan stock, los pagos confirmados se realizan sin disponibilidad real y las notificaciones de disponibilidad no siempre llegan a los usuarios que solicitaron alertas. Estos problemas generan inconsistencias en la plataforma, afectando la fiabilidad del servicio y la experiencia del usuario.

A partir de este escenario, nuestra tarea es identificar y justificar un patrón adecuado para gestionar la gestión distribuida de transacciones dentro de nuestra plataforma, considerando las restricciones acordadas, y analizar sus beneficios y desventajas.

Los problemas que se han detectado durante las pruebas, como pagos cobrados sin stock real, stock que no se libera al cancelar y notificaciones perdidas demuestran una falta de consistencia de datos entre múltiples microservicios.

Para resolver este problema, el patrón más adecuado es el patrón Saga (*Saga pattern*), concretamente la variante **Saga orquestada** (*Orchestration-based Saga*), ya que este patrón gestiona la consistencia de los datos a través de múltiples microservicios sin utilizar transacciones distribuidas tradicionales como el Two-Phase Commit o 2PC, el cual bloquea los recursos y afecta al rendimiento.

Una Saga es una secuencia de transacciones locales. Cada servicio local actualiza su base de datos y publica un evento o responde a un comando para activar el siguiente paso de la Saga. Si una transacción local falla, como por ejemplo, si el usuario ya pagó pero el catálogo dice que no hay stock, la Saga ejecuta una serie de transacciones compensatorias para deshacer los cambios realizados por las transacciones locales anteriores. En el caso del ejemplo, podría ordenar al sistema de pagos que devuelva el dinero.

?

Richardson define dos formas de coordinar Sagas. Hemos elegido la **orquestración** porque el proceso de alquiler de Koursera involucra a más de 3 servicios y tiene una lógica condicional compleja. En la otra forma, la cual es *coreografía*, los servicios reaccionan a eventos de otros, similar a un efecto dominó. Esto es útil para flujos simples de 2 pasos, pero para un proceso de checkout crearía un acoplamiento cíclico difícil de rastrear y depurar.

En la Orquestración, se crea una clase central (*orchestrator*) que actúa como un director de orquesta. Sabe exactamente qué paso toca, espera las respuestas y, si algo falla, sabe exactamente a quién llamar para deshacer los cambios.

Podemos identificar numerosos beneficios al usar este patrón. Uno de ellos es la consistencia sin bloqueos, ya que mantiene la consistencia de los datos en todo el sistema sin mantener bloqueos prolongados en las bases de datos de Payment o Catalog, lo cual asegura una alta disponibilidad y escalabilidad. Por otra parte, también presenta resiliencia ante fallos, dado que resuelve directamente el problema del CTO: si el alquiler se cancela o falla la reserva en el catálogo, la Saga devuelve el dinero y el estado del alquiler se marca como fallido para evitar estados inconsistentes.

Por último, la lógica centralizada hace que sea mucho más fácil entender el flujo de negocio mirando el código del orquestador que intentando rastrear eventos disparados por toda la plataforma.

Como contraparte, identificamos un aumento considerable en la complejidad del desarrollo: Por cada paso de la transacción, los desarrolladores deben programar explícitamente su contraparte compensatoria (*success/fail paths*). También se suma la falta de aislamiento, ya que mientras la Saga se está ejecutando, otros servicios pueden ver estados parciales, lo cual requiere diseñar el dominio con cuidado usando estados intermedios como PENDING o RESERVED.